

CHAPTER III

METHODOLOGY

This chapter presents the technical background, requirements analysis and documentation, design of software, system technicalities, and development method used for TASKLINK.

Technical Background

TASKLINK is a mobile job matching system designed to connect clients who need short-term local assistance with informal workers who are available to perform nearby tasks in Bacolod City. The system focuses on mobile-based task posting, worker application, client approval, task status monitoring, in-app communication, payment proof recording, and rating or feedback after task completion.

The application was developed as an Expo React Native project for Android devices. Its routing structure is implemented through Expo Router, while shared application data and actions are managed through the React Context API. The system uses Firebase services for its backend operations. Firebase Identity Toolkit REST calls are used for registration and login, Cloud Firestore is used for real-time application data, and Firebase Storage is initialized for possible file storage requirements such as profile and verification document references. The project also includes EAS Build configuration for Android development, preview, and production builds.

Based on the project files, the system supports two active user roles: worker and client. Workers can register, maintain a profile, set capabilities and service area information, browse available jobs, apply to tasks, track task progress, chat with clients, mark work as finished, and rate clients. Clients can register, post tasks, review applicants, accept workers, track ongoing tasks, submit payment proof, confirm completion, archive tasks, chat with workers, and rate workers. An admin role is mentioned as unclear in the documentation, but no admin module, route, dashboard, or permission system was found in the project files.

Technologies to be Used

The technologies used in the development of TASKLINK were selected to support rapid mobile application development, real-time database updates, and cloud-backed authentication. Expo and React Native serve as the main application framework, while Firebase provides the backend services needed by the system.

Technology	Purpose in the System
Expo	Provides the mobile application runtime and development platform.
React Native	Used to build the Android mobile user interface.
React	Provides the component-based UI foundation of the application.
TypeScript	Adds static typing and improves code reliability.
Expo Router	Handles file-based navigation through the application screens.
React Context API	Manages shared application state, user session, tasks, notifications, messages, ratings, and actions.
Firebase JS SDK	Initializes Firebase app, Firestore, and Storage services.
Firebase Identity Toolkit REST API	Handles registration and login using mobile-number-based email accounts.
Cloud Firestore	Stores users, worker profiles, client profiles, tasks, task matches, payments, chats, messages, ratings, and notifications.
Firebase Storage	Initialized for storage support; actual upload handling is not fully implemented in the current project files.
Expo Notifications	Installed as a dependency; current implementation stores in-app notifications in Firestore rather than sending real push notifications.
EAS Build	Provides Android build profiles for development, preview, and production.
React Native Safe Area Context	Helps the interface adapt to safe screen areas on mobile devices.

The application is configured with the name TASKLINK, portrait orientation, light user interface style, and Android package com.tasklink.app. For capstone deployment, the system is treated as an Android mobile application for Android device users.

Calendar of Activities

The actual project files do not contain an official development calendar, Gantt chart, or exact activity completion dates. Therefore, the following tables present a proposed calendar of activities aligned with the project scope and with the academic semesters required in the capstone format.

Table 1

Activity	Purpose/Objectives	Persons Involved	Resources Used
Problem Identification and Initial Proposal	Identified the problem of unreliable word-of-mouth hiring for informal/manual labor in Bacolod City and proposed TASKLINK as a mobile job matching solution.	Researchers, adviser, target user representatives	Capstone guide, project documentation, preliminary requirements
Requirements Gathering	Identified the target users, including workers and clients, and defined the major system needs such as task posting, worker application, communication, and rating.	Researchers, potential workers, potential clients	Requirements notes, documentation files, project source review
System Analysis	Analyzed task data, user roles, payment method requirements, notification needs, task lifecycle, and trust-related features.	Researchers, system analyst/developer	src/types.ts, src/context/AppContext.tsx, service modules
Prototype Planning	Selected Expo, React Native, TypeScript, Firebase, and mobile-first architecture as the development foundation.	Researchers, mobile developer	package.json, app.json, Firebase configuration module
UI/UX Wireframing and Prototype Design	Planned the required screens such as splash, login, dashboards, job board, task details, task status, chat, rating, and profile screens.	Researchers, UI/UX designer, developer	app/ route structure, reusable component plan

Table 2

Activity	Purpose/Objectives	Persons Involved	Resources Used
System Development	Implemented the Expo Router screens, reusable components, and mobile user interface.	Mobile developer, UI/UX designer, researchers	Expo, React Native, TypeScript, app/, src/components/

Activity	Purpose/Objectives	Persons Involved	Resources Used
Backend Integration	Integrated Firebase configuration, Firebase Authentication REST calls, and Firestore service modules.	Backend/Firebase developer, mobile developer	Firebase, Firestore, src/services/
Feature Implementation	Developed registration, login, role selection, task posting, worker application, task tracking, chat, rating, payment proof, notifications, and profile management.	Developers, researchers	src/context/AppContext.tsx, route files, service modules
Testing and Refinement	Conducted TypeScript checking and manual workflow validation. Automated test files were not found in the current project.	QA tester, developers, researchers	npm run typecheck, Android device/emulator, manual test scenarios
Documentation and Deployment Preparation	Prepared project documentation and configured EAS Build profiles for Android development, preview, and production builds.	Documentation writer, developer, researchers	docs/, eas.json, project source files

Resources

The development and operation of TASKLINK require hardware, software, and human resources suitable for mobile application development and Firebase-backed cloud services.

Hardware Recommendation

The project files do not contain formal hardware specifications. The following recommendations are based on the actual technology stack and user workflows found in the application.

Table 3

Hardware Requirements

User/Role	Recommended Hardware	Purpose
Developer	Laptop or desktop computer capable of running Node.js, Expo tooling, TypeScript, and Android development tools	Used for coding, running the app, type checking, and preparing builds.
Worker User	Android smartphone with reliable internet connection	Used to register, browse jobs, apply, chat, track tasks, and submit ratings.
Client User	Android smartphone with reliable internet connection	Used to post tasks, review applicants, chat, submit payment proof, confirm completion, and rate workers.
Network Resource	Stable internet connection	Required for Firebase Authentication REST calls and Firestore real-time data synchronization.
Server Side	Firebase cloud infrastructure	Custom server hardware is not specified because the current system uses Firebase cloud services.

Server hardware is not specified because the current system relies on Firebase cloud services rather than a custom local or dedicated backend server.

Software Recommendation

The software resources required for TASKLINK include development tools, build tools, and cloud service configuration.

Table 4

Software Requirements

Software	Purpose
Node.js and npm	Used to install dependencies and run project scripts.
Expo CLI / Expo Tooling	Used to start and test the Expo React Native application.
EAS CLI	Used to create Android development, preview, and production builds.
TypeScript	Used for type checking and code safety.
Firebase Project	Provides authentication, Firestore database, and initialized storage services.

Software	Purpose
Android Device or Emulator	Used for mobile testing and deployment validation.
Visual Studio Code or Equivalent IDE	Used as the development environment.

The Firebase configuration requires public Expo environment variables for API key, authentication domain, project ID, storage bucket, messaging sender ID, and app ID. These values are expected by the Firebase configuration module and must be provided for the application to use Firebase-backed data.

Human Resources Recommendation

The project files do not define an official team roster. The following human resource recommendations are based on the system scope and the technical work required to develop and document TASKLINK.

Table 5

Human Resources

Role	Responsibility
Project Manager/Researcher	Coordinates the capstone schedule, requirements analysis, documentation, and evaluation activities.
Mobile Developer	Develops the Expo React Native screens, navigation, UI logic, and shared components.
Backend/Firebase Developer	Configures Firebase Authentication, Firestore collections, Firebase Storage initialization, and service modules.
UI/UX Designer	Designs mobile workflows suitable for informal workers and clients.
Quality Assurance Tester	Tests registration, login, task posting, job application, task status, chat, payment proof, rating, and profile workflows.
Documentation Writer	Prepares capstone documents, system diagrams, methodology discussion, and supporting technical descriptions.

Requirements Analysis

The PIECES Framework was used to analyze the system requirements of TASKLINK. This framework examines the system in terms of performance, information and data, economics, control and security, efficiency, and service.

Table 5

PIECES Framework

PIECES Area	Current Problem or Need	TASKLINK Response
Performance	Manual hiring through word-of-mouth is slow and unreliable. Workers need fast access to available jobs, while clients need to review applicants quickly.	The system uses Firestore subscriptions to load updated users, tasks, messages, ratings, and notifications. Dashboards display current jobs and applications for faster decision-making.
Information and Data	Workers and clients need structured information about tasks, profiles, ratings, status, payment, and messages.	TASKLINK defines structured data models for users, tasks, matches, messages, ratings, notifications, and payments. Screens display task category, location, wage, duration, status, applicants, capabilities, and ratings.
Economics	Informal workers need access to earning opportunities, while clients need a simple way to offer wages for local help.	Clients can enter wage offers and select either COD or GCash link as payment method. The app records payment status and proof text, although real payment gateway integration is not implemented.
Control and Security	The system must protect user actions, prevent duplicate applications, separate user roles, and validate important inputs.	Firebase Authentication REST calls support account access. AppContext validates logged-in users and role-based actions. Firestore transactions prevent duplicate applications and conflicting assignment updates.
Efficiency	Users need fewer steps to post, find, apply for, monitor, and complete tasks.	Worker and client dashboards, quick apply, task status buttons, bottom navigation, and reusable UI components support a more efficient workflow.
Service	The system must support communication, updates, trust, and	TASKLINK includes in-app chat, notification records, ratings, worker verification fields,

PIECES Area	Current Problem or Need	TASKLINK Response
	confirmation between workers and clients.	payment proof, and two-sided task completion flow.

Ethical Considerations

The development of TASKLINK involves ethical concerns related to personal data, fair access, user trust, and privacy. The system stores user-related information such as full name, mobile number, address or location, role, skills or capabilities, ratings, profile photo reference, valid ID reference, medical certificate reference, and worker coordinates. Since these data may identify users, the system should be handled with confidentiality and should be used only for legitimate task matching and profile verification purposes.

The application is intended for informal workers and clients in Bacolod City. Therefore, the language and design of the system should respect the dignity of informal workers and avoid discriminatory or stigmatizing descriptions. The worker and client roles must be presented as legitimate participants in a local service marketplace. Workers should be given clear information about tasks, wage offers, client communication, and task expectations before accepting or performing work.

The project includes worker verification fields such as valid ID type, valid ID reference, medical certificate reference, profile photo reference, and experience description. However, an actual admin review or verification workflow was not found in the current project files. For ethical implementation, these fields should not be treated as fully verified unless a proper review process is implemented. The application also shows terms and privacy references in the user interface, but formal legal terms and privacy policy documents were not found in the project files. These documents should be prepared before production deployment.

Payment in the current system is represented through COD or GCash link options and payment proof text. The system does not process real payments through a payment gateway. For ethical use, the system should clearly inform users that payment arrangements are external or manually verified. The location and geofence features should also be treated carefully because location-related data can affect user privacy. The current project uses coordinate and radius checks for task status actions, but live GPS tracking was not found in the project files.

Requirements Documentation

The current project files show the following system features:

- Feature 1: Splash Screen — Displays TASKLINK branding, tagline, illustration, loading animation, and redirects users to the login screen.
- Feature 2: Login by Mobile Number — Allows users to enter a mobile number and authenticate through Firebase Identity Toolkit using a normalized mobile-number email format.
- Feature 3: Prototype OTP-Style Login UI — Provides an OTP-style interface with send-code and verification-code fields, but real SMS OTP verification is not implemented.
- Feature 4: Password Login — Allows users to log in with mobile number and password.
- Feature 5: Mobile Registration — Allows account creation using full name, mobile number, password, address or barangay, and worker capabilities.
- Feature 6: Role Selection — Allows users to choose between worker and client roles and routes them to the appropriate dashboard.
- Feature 7: Worker Dashboard — Displays worker availability, available jobs, applications, finished jobs, job highlights, and navigation options.
- Feature 8: Client Dashboard — Displays client task statistics, ongoing jobs, completed jobs, archived tasks, active workers, total spend, and post-task entry points.
- Feature 9: Worker Job Board — Displays available jobs, applied tasks, filters, distance indicators, service radius indicators, capability match indicators, and quick apply actions.
- Feature 10: Post Task — Allows clients to enter task category, title, location, service area, coordinates, geofence radius, required capability, wage, estimated duration, payment method, and notes.
- Feature 11: Task Creation in Firestore — Saves task documents with a default Finding Workers status and creates linked payment records.
- Feature 12: Worker Application — Allows workers to apply to open tasks, stores task match records, updates applicant IDs, and changes task status to Applied.
- Feature 13: Applicant Review — Allows clients to view worker applications and accept a worker from the task details screen.
- Feature 14: Worker Public Profile — Displays worker capabilities, ratings, verification fields, document completion indicators, service coverage, and accept or message actions when connected to a task.
- Feature 15: Task Status Tracking — Displays task status, location context, task details, employer/client information, location check, payment verification, and primary status

actions.

- Feature 16: Task Lifecycle Management — Supports Finding Workers, Applied, Accepted, In Progress, Pending Approval, Finished, and Archived statuses.
- Feature 17: Worker Start and Finish Controls — Allows accepted workers to start a task and mark it as finished, subject to location/geofence checks.
- Feature 18: Client Completion Approval — Allows the client to confirm a worker-finished task and archive a finished task.
- Feature 19: Location and Geofence Check — Calculates the distance between worker and task coordinates and checks whether the worker is inside the allowed task radius before selected status actions.
- Feature 20: Worker Service Area Preferences — Allows workers to set availability, location area, coordinates, and preferred service radius.
- Feature 21: Capability Matching — Stores worker capabilities and task-required capability, then displays capability match indicators on the job board.
- Feature 22: In-App Chat Inbox — Lists task conversations with search, participant names, previews, timestamps, and unread indicators.
- Feature 23: Task Chat — Allows task participants to send messages, use quick replies, and view message bubbles in a task-based conversation.
- Feature 24: Notification Records — Stores and displays in-app notifications for nearby tasks, worker applications, accepted applications, completion approval, and payment proof submission.
- Feature 25: Rating and Feedback — Allows users to rate the other party after a task, stores ratings, and updates the target user's aggregate rating.
- Feature 26: Profile Management — Allows users to edit full name, mobile number, address, bio or business name, worker skills, verification fields, availability, coordinates, service radius, and logout.
- Feature 27: Worker Verification Fields — Stores worker verification status, profile photo reference, experience, years of experience, valid ID type/reference, and medical certificate reference.
- Feature 28: Payment Method Selection — Allows clients to select COD or GCash link when posting a task.
- Feature 29: Payment Proof Submission — Allows clients to submit payment proof text or reference and notifies the assigned worker.

- Feature 30: Firestore Real-Time Subscriptions — Subscribes to users, tasks, messages, ratings, and notifications.
- Feature 31: Reusable UI Components — Provides buttons, cards, inputs, bottom navigation icons, empty states, screen container, and status badges.
- Feature 32: Android Build Profiles — Provides EAS build profiles for development, preview, and production Android builds.

Admin functionality, real SMS OTP provider integration, real device push notifications, real payment gateway integration, live GPS tracking, account recovery, account deletion, and automated tests were not found in the current project files.

Design of Software, System, Product, and/or Processes

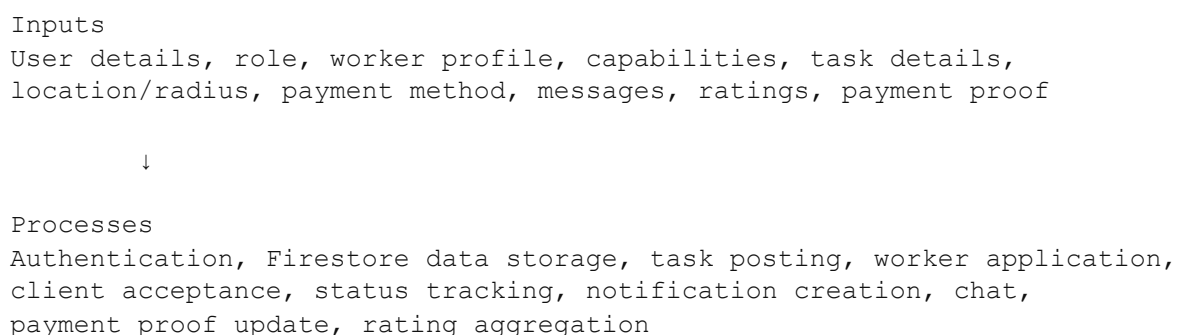
Project Framework

TASKLINK follows an input-process-output framework. The inputs include user registration details, login credentials, role selection, worker profile information, worker capabilities, service area details, task details, location and radius values, payment method, chat messages, rating scores, feedback, and payment proof. These inputs are processed through the mobile application, AppContext action layer, Firebase Authentication REST calls, Firestore service modules, task lifecycle logic, notification creation, chat handling, payment proof updates, and rating aggregation.

The expected outputs are role-based dashboards, available job listings, posted task records, applicant lists, task status updates, in-app notifications, chat conversations, payment status displays, rating records, updated user ratings, and archived task records.

Figure 3

Project Framework of TASKLINK



↓
Outputs Dashboards, job board, task status, applicant list, notifications, chat records, ratings, payment status, archived tasks

On the Design of Software

The software design of TASKLINK is organized around mobile screens and reusable components. The route files under the app/ folder define the major screens, while shared interface components are stored in src/components/. The root layout wraps the application with the AppProvider, SafeAreaProvider, status bar, and stack navigation.

Figure	Screen or Component	Expected Output
Figure 4	Login and Registration Forms	Provides OTP-style login UI, password login, inline registration, and account creation inputs.
Figure 5	Role-Based Dashboard Forms	Shows the worker dashboard and client dashboard outputs based on selected role.
Figure 6	Task Posting and Job Board Forms	Allows clients to post tasks and workers to view available jobs with matching indicators.
Figure 7	Task Details and Task Status Forms	Shows task information, applicants, lifecycle controls, location checks, payment proof, chat link, and rating link.
Figure 8	Communication, Rating, Profile, and Notification Forms	Displays chat, ratings, profile management, worker public profile, and notifications.

Figure 4

Login and Registration Forms

The login and registration forms allow users to access the system through a mobile number and password. The login screen also provides an OTP-style prototype interface, while the registration inputs gather full name, mobile number, address or barangay, password, and worker capabilities.

Figure 5

Role-Based Dashboard Forms

The worker dashboard presents available jobs, active applications, finished jobs, and quick navigation. The client dashboard presents task statistics, ongoing jobs, completed tasks, archived

tasks, active workers, and post-task access.

Figure 6

Task Posting and Job Board Forms

The post-task form allows clients to enter task details, location, radius, wage, duration, required capability, and payment method. The job board displays available tasks, matching details, worker service radius information, and quick apply actions.

Figure 7

Task Details and Task Status Forms

The task details form displays the job information, applicants, worker profile access, apply action, and accept action. The task status form displays the task lifecycle, location check, payment proof, chat link, rating link, and status control buttons.

Figure 8

Communication, Rating, Profile, and Notification Forms

The communication and support forms include the chat inbox, task chat, rating and feedback screen, user profile screen, worker public profile screen, and notification list.

System Functionalities

The system functionalities of TASKLINK are represented through the context diagram, data flow diagrams, and user interaction flowcharts. These diagrams describe the movement of information between users, system processes, and data stores. They also show how worker and client actions are transformed into expected system outputs.

Figure 9

Context Diagram

The context diagram should show TASKLINK as the central system. The external entities are Worker, Client, and Firebase Services. Workers send registration data, login data, profile updates, applications, messages, task status updates, and ratings to the system. Clients send registration data, login data, task posts, applicant decisions, payment proof, messages, completion confirmations, archive actions, and ratings. Firebase Services provide authentication, Firestore data storage, real-time synchronization, and storage initialization.

Figure 10

Level 1 Data Flow Diagram of TASKLINK

The Level 1 DFD should contain the following major processes: User Authentication and Role Management, Profile and Verification Management, Task Posting and Job Discovery, Worker Application and Client Acceptance, Task Status and Completion Management, Chat and Notifications, Payment Proof Management, and Rating and Feedback Management. The data stores should include Users/Profile Store, Task Store, Match Store, Payment Store, Message Store, Rating Store, and Notification Store.

Figure 11

Level 2 Data Flow Diagram of User Authentication and Role Management

For authentication and role management, the system receives mobile number, password, full name, address, and role. It normalizes the mobile number, uses Firebase Identity Toolkit for sign-up or sign-in, stores or reads the user profile from Firestore, updates the role when needed, and routes the user to the correct dashboard.

Figure 12

Level 2 Data Flow Diagram of Task Posting and Matching

For task posting, the system receives title, description, category, location, coordinates, geofence radius, required capability, wage, duration, and payment method. It validates required fields, creates a Firestore task record, creates a payment record, and sends worker notification records.

For job application and matching, the worker applies to an open task. The system checks task existence, duplicate application, assignment status, and open status through a Firestore transaction. It then creates a task match record, updates applicant IDs, changes the task status to Applied, and notifies the client.

For task progress, the system receives status actions from the assigned worker or client. It updates the task status and timestamps, enforces geofence checks for worker start and finish actions, updates worker availability after completion, and displays the new status on dashboards and task status screens.

For chat, the system receives message text, sender ID, receiver ID, and task ID. It updates chat metadata, stores the message document, and displays messages in the chat inbox and task chat screen.

For notifications, the system receives trigger events such as task posting, worker application, accepted application, completion approval, and payment proof submission. It creates notification

records and displays them to the affected user.

For payment proof, the system receives the payment method and proof/reference text. It updates the task and payment records, changes the payment status, and notifies the worker when proof is submitted.

For rating, the system receives score and feedback. It creates a rating record and updates the target user's rating count, rating total, and average rating.

Figure 13

User Interaction Flowchart for Worker

Worker flow:

1. Open TASKLINK.
2. View splash screen.
3. Log in or register.
4. Select Worker role if needed.
5. Open Worker Dashboard.
6. Open Jobs or view available job cards.
7. View job details.
8. Apply or Quick Apply.
9. Wait for client acceptance.
10. Open Task Status.
11. Start Task after acceptance and location check.
12. Mark task as finished.
13. Wait for client completion approval.
14. Chat with the client as needed.
15. Rate the client after completion.
16. View finished jobs or update profile.

Figure 14

User Interaction Flowchart for Client

Client flow:

1. Open TASKLINK.
2. View splash screen.

3. Log in or register.
4. Select Client role if needed.
5. Open Client Dashboard.
6. Post a task.
7. Review applicants in Task Details.
8. View worker profile.
9. Accept worker.
10. Chat with the worker as needed.
11. Monitor Task Status.
12. Submit payment proof.
13. Confirm task as finished after worker completion.
14. Rate the worker.
15. Archive the completed task.

Figure 15

User Interaction Flowchart for Admin

Admin flow was not found in the project files because the source code only implements worker and client roles. If an admin module is required by the final capstone scope, its dashboard, permissions, moderation processes, and database controls must be specified separately before Figure 15 can be completed.

System Technicalities

On Network Architecture

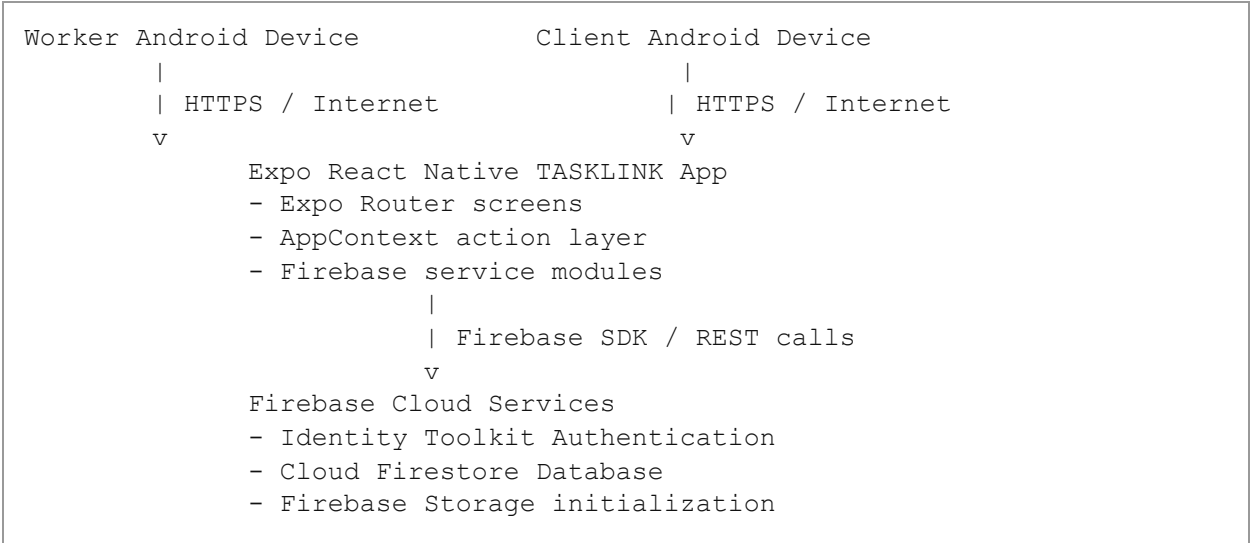
TASKLINK uses a cloud-backed mobile network architecture. The mobile client is an Expo React Native application running mainly on Android devices. The application communicates with Firebase services over the internet. Firebase Identity Toolkit handles account registration and login, while Firestore stores and synchronizes application data such as users, tasks, task matches, payments, chats, messages, ratings, and notifications. Firebase Storage is initialized for file storage readiness, although complete file upload handling was not found in the current implementation.

The application code is organized into presentation, context, and service layers. Screens under the app/ directory provide the user interface and navigation. The AppContext layer manages

session state, data arrays, loading and error states, and system actions. The service layer under src/services/ communicates with Firebase and Firestore.

Figure 16

Network Architecture of TASKLINK



No custom backend server, local server hardware, Cloud Functions, REST API routes, or Firestore security rules file was found in the current repository.

On Access Control

The access control design of TASKLINK is based on role-specific workflows. The implemented front-end roles are worker and client. Workers are directed to worker-specific screens and actions, while clients are directed to client-specific task management workflows. Protected actions check the current user session before continuing. Some actions also check the user's role, such as payment proof submission, which is limited to clients.

Table 6

Access Profile for Back-end User

Back-end/User Type	Access or Permission
Firebase-configured App Service Layer	Reads and writes Firestore data through service modules when Firebase environment variables are configured.
Authenticated Worker/Client through AppContext	Performs Firestore actions through context functions after login and profile loading.

Back-end/User Type	Access or Permission
Admin	Not found in project files.
Manual Database Administrator	Not represented in application code. Possible Firebase console operation is outside the current project files.

Table 7

Access Profile for Front-end User

Front-end User	Allowed Access or Permission
Worker	Register, log in, choose worker role, update profile, set capabilities and service area, browse jobs, apply to tasks, track accepted tasks, start task, mark task as finished, chat with client, rate client, and view notifications.
Client	Register, log in, choose client role, update profile or business bio, post tasks, review applicants, view worker profiles, accept worker, track tasks, submit payment proof, confirm completion, archive task, chat with worker, rate worker, and view notifications.
Guest/Unauthenticated User	Can access splash and login screens. Protected actions require login and show errors if the user is not authenticated.
Admin	Not found in project files.

On Data Security and Controls

TASKLINK uses Firebase configuration values loaded from Expo public environment variables. The Firebase app initializes only when all required values are available. Authentication is handled using Firebase Identity Toolkit REST endpoints. Registration uses the signUp endpoint, and login uses the signInWithPassword endpoint. Mobile numbers are normalized into local email-style accounts using the format 639XXXXXXXXXX@tasklink.local.

The system includes validation and control logic in its shared context and service modules. Registration checks that the full name is present and that the mobile number contains enough digits. Task creation checks that the title and location are provided and that the wage is valid. Coordinate parsing returns undefined for invalid coordinate values. Firestore transactions are used to prevent duplicate applications, prevent application to assigned or closed tasks, update task status safely, and recalculate rating averages.

Role-based controls are applied through the AppContext action layer. Actions that require a logged-in user throw readable errors when no current user exists. Client-specific actions, such as submitting payment proof, require a client role. Worker start and finish actions use location and geofence checks. Chat receiver selection is based on task participants, applicant IDs, worker ID, and client ID.

The system also includes basic error handling. AppContext stores error, app loading, and action loading states. Screens use try/catch blocks and display readable errors to users. Firestore listener errors are passed to a listener error handler. If Firebase is not configured, service modules return empty data for subscriptions or throw readable Firebase configuration errors for actions requiring Firestore.

Despite these controls, several security-related items were not found in the current project files. These include Firestore security rules, real SMS OTP enforcement, account recovery, account deletion, rate limiting, anti-spam controls, audit logs, real legal privacy policy pages, and admin moderation controls.

Development

Research Method

The most suitable research method for TASKLINK is the descriptive-developmental research method. The descriptive part applies because the study identifies and describes an existing problem in the local hiring process for informal/manual labor in Bacolod City. The developmental part applies because the project produces a mobile application as a proposed solution to that problem.

This method is appropriate because TASKLINK is not only a study of a problem but also a system development project. The application demonstrates how clients can post tasks, how workers can apply, how both users can communicate, how task status can be tracked, and how ratings can support trust after a completed transaction. Survey results, interview results, or formal user evaluation data were not found in the project files; therefore, these should not be claimed unless provided separately.

System Development Method

The most suitable system development method for TASKLINK is an Agile or iterative prototype development method. The project files show an evolving mobile prototype that began with earlier prototype documentation and now includes Firebase-backed services, route-based screens,

task workflows, chat, ratings, notifications, payment proof recording, and profile management. This indicates that the system is suited to iterative development, where screens and features can be built, tested, refined, and integrated in stages.

Phase 1. Planning, Requirements Analysis, and Prototype Design

The first phase involves identifying the target users and defining the major system requirements. For TASKLINK, the main target users are workers and clients. The major workflows include registration, login, role selection, profile management, task posting, job application, applicant review, task status tracking, chat, payment proof, rating, and notifications. During this phase, the system data model and route structure are also prepared.

Phase 2. System Development and Integration

The second phase involves building the mobile screens, reusable components, context actions, and Firebase service modules. This includes implementing the worker dashboard, client dashboard, job board, task posting screen, task details screen, task status screen, chat screens, rating screen, profile screen, notifications screen, and worker public profile screen. Firebase Authentication REST calls and Firestore service modules are integrated to support real-time data and user workflows.

Phase 3. Testing, Refinement, and Deployment Preparation

The third phase involves checking the TypeScript project, manually validating user workflows, refining screens and validation, and preparing deployment builds. The project includes an npm run typecheck script and EAS Build profiles for development, preview, and production Android builds. Automated unit tests, integration tests, end-to-end tests, formal usability results, and user acceptance testing documents were not found in the current project files.

Figure 17

System Development Method

